

Competitor Pricing Analysis

Supabase vs Firebase vs Appwrite: pricing, offerings and user engagement

Internship project · Farwa Eeman · Data collected 2-3 October 2026

\$25

per month: first paid plan at Supabase and Appwrite

\$0

fixed monthly fee at Firebase (pay for usage only)

2,210 GB

stored: above this size Supabase becomes the cheapest

111,029

GitHub stars for Supabase, the most of the three

Overview

Objective: analyze competitors' services and pricing to get market insights. The project collects pricing, feature and engagement data for three competing platforms, compares them with charts and cost scenarios, and turns the results into pricing recommendations.

Assumption

The task did not name an industry or competitors, and there was no supervisor to ask. I chose three backend platforms for app developers: **Supabase, Firebase and Appwrite**.

How the data was collected

- **Engagement:** GitHub stars, forks and open issues, from the public GitHub API.
- **Pricing and offerings:** I checked each site's robots.txt, downloaded the pricing pages with requests and BeautifulSoup, and searched the text for prices. I then confirmed plan names, prices and limits by reading the pages in the browser and entered them into tables.
- **Appwrite:** its Free and Pro plan details were read in the browser, because the downloaded page text only contained database prices.

Key findings

- **Pricing models differ.** Supabase and Appwrite start at \$25 per month. Firebase has no fixed fee and charges for usage only.
- **The cheapest platform depends on app size.** Firebase is cheapest below about 110 GB stored, Appwrite from about 110 to 2,200 GB, and Supabase above about 2,200 GB.
- **Appwrite includes the most** in its Pro plan: 150 GB storage, 2 TB data transfer and 200K active users.
- **Supabase has the most developer interest**, with about 111,000 GitHub stars, followed by Appwrite with about 57,500.

1 Pricing

Plan fees

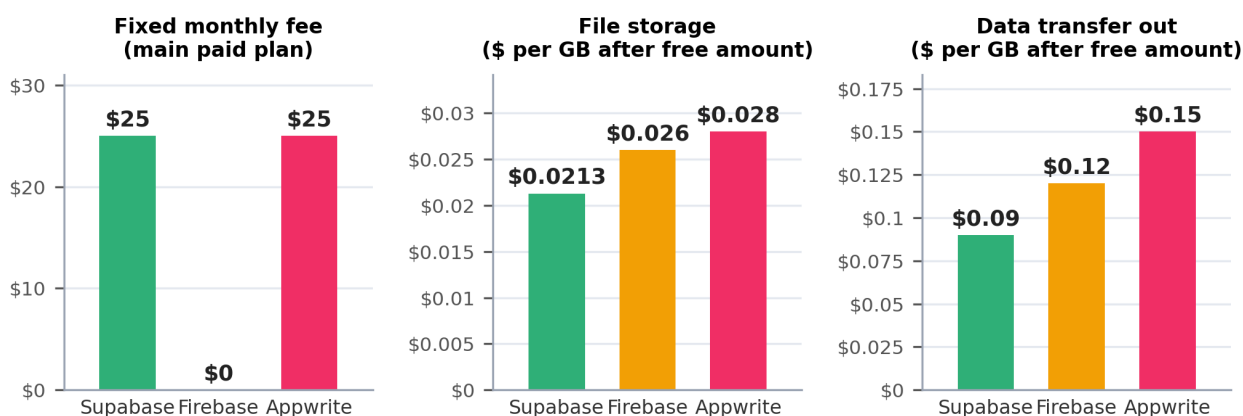
Platform	Plan	Monthly fee	Pricing model
Supabase	Free	\$0	Fixed plan fee
Supabase	Pro	\$25	Fixed plan fee + usage
Supabase	Team	\$599	Fixed plan fee + usage ('from' price)
Firebase	No-cost (Spark)	\$0	Free with fixed limits
Firebase	Pay as you go (Blaze)	\$0	No fixed fee, pay for usage
Appwrite	Free	\$0	Free with fixed limits
Appwrite	Pro	\$25	Fixed plan fee ('from' price) + pay as you go

Pro and Team prices are starting ("from") prices on the pricing pages.

Usage prices

Platform	Item	Included free	Price after free amount
Supabase	File storage	100 GB (Pro)	\$0.0213 per GB
Firebase	File storage (Cloud Storage, legacy buckets)	5 GB (Blaze plan)	\$0.026 per GB
Appwrite	File storage	150 GB (Pro)	\$0.028 per GB
Supabase	Data transfer out	250 GB (Pro)	\$0.09 per GB
Firebase	Data transfer out (Cloud Storage downloads)	1 GB/day (Blaze plan)	\$0.12 per GB
Appwrite	Data transfer out (API bandwidth)	2 TB (Pro)	\$0.15 per GB
Supabase	Active users (MAU)	100,000 (Pro)	\$0.00325 per user
Firebase	Active users (Authentication)	50,000 (some features)	Not shown on page
Appwrite	Active users (MAU)	200,000 (Pro)	Not shown on page

Free amounts are for the main paid plan: Supabase Pro, Appwrite Pro and Firebase Blaze.



Key takeaway

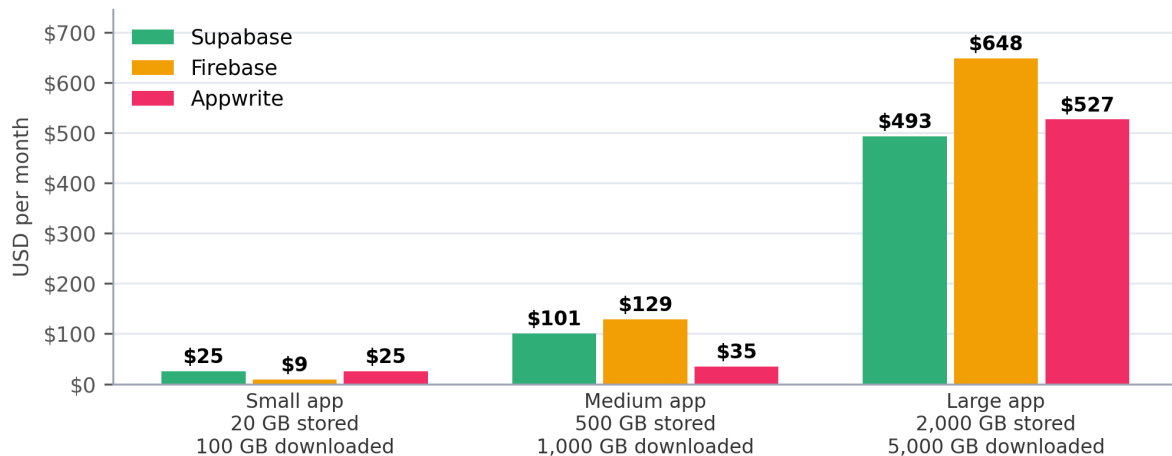
Supabase and Appwrite both start at **\$25 per month**. Firebase has no fixed fee. For extra usage, **Supabase is cheapest** for file storage (\$0.0213 per GB) and data transfer (\$0.09 per GB), then Firebase (\$0.026 and \$0.12), then Appwrite (\$0.028 and \$0.15).

2 Cost scenarios and cost curve

Monthly cost is the plan fee plus the charges above the included storage and data transfer. Database, compute and extra user costs are not included.

App size	Stored	Downloaded	Supabase	Firebase	Appwrite
Small app	20 GB	100 GB	\$25.00	\$8.79	\$25.00
Medium app	500 GB	1,000 GB	\$101.02	\$129.27	\$34.80
Large app	2,000 GB	5,000 GB	\$492.97	\$648.27	\$526.80

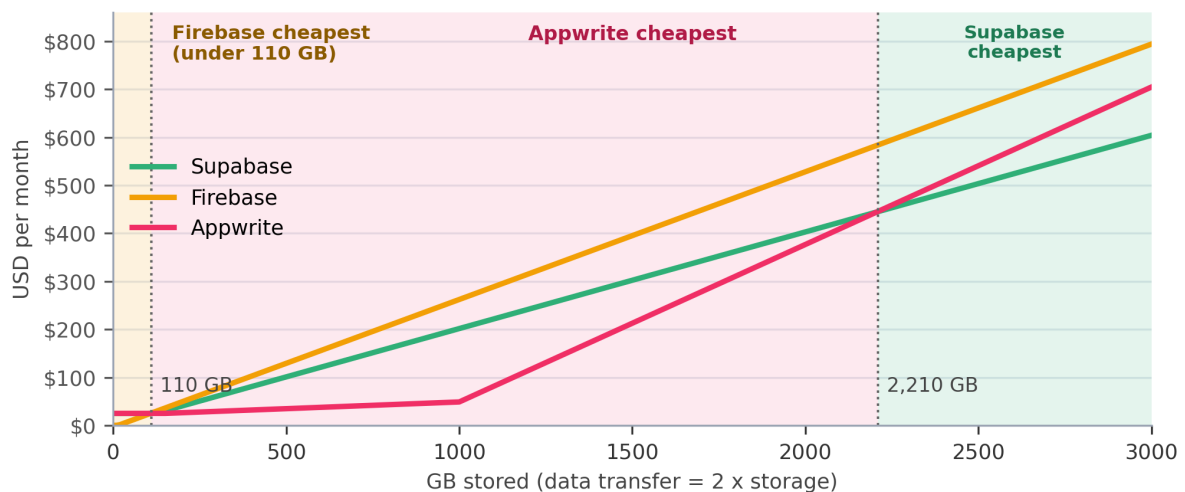
The cheapest platform in each row is highlighted in green. The three app sizes are examples I chose.



Key takeaway

Firebase is cheapest for the small app because it has no fixed fee. **Appwrite** is cheapest for the medium app because its Pro plan includes 2 TB of data transfer. **Supabase** is cheapest for the large app because it charges the least for extra data transfer.

Cost curve

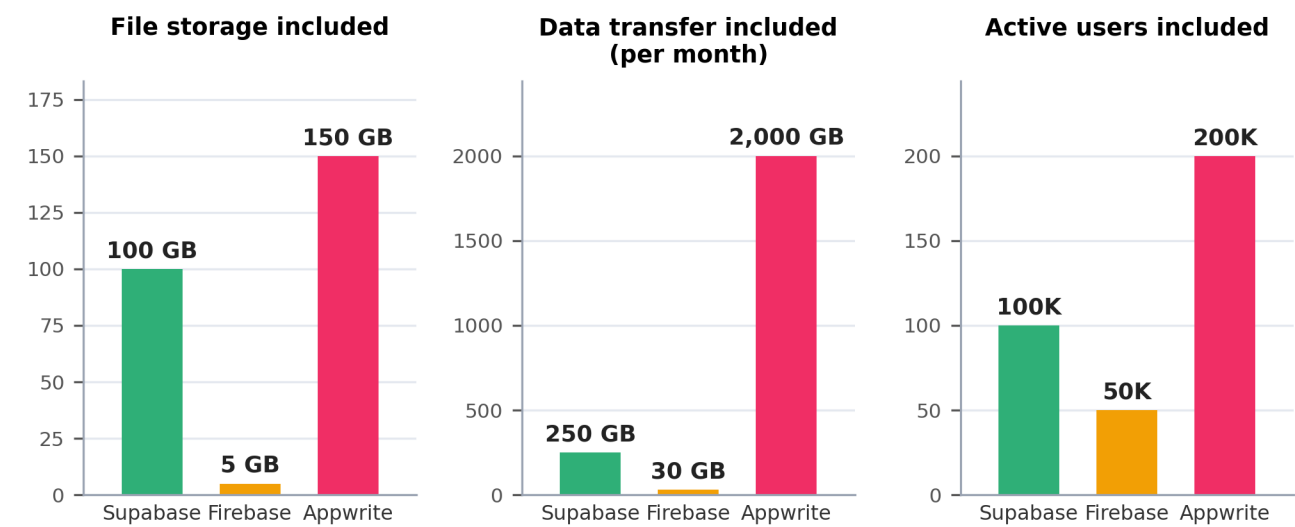


Key takeaway

The cheapest platform changes at **110 GB** (Firebase to Appwrite) and at **2,210 GB** (Appwrite to Supabase). This assumes the app downloads twice as much data as it stores.

3 Offerings: included resources, features and free plans

Included in the main paid plans



Key takeaway

Appwrite Pro includes the most storage, data transfer and active users. Firebase's 30 GB of data transfer is an approximation of its free 1 GB per day.

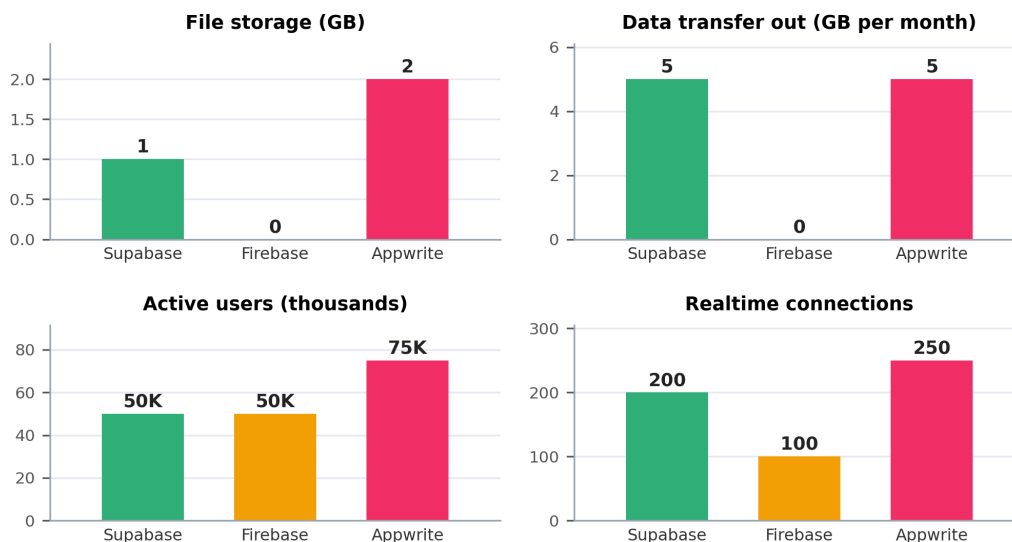
Features listed on each pricing page

	Supabase 5 of 9 listed	Firebase 9 of 9 listed	Appwrite 8 of 9 listed
Authentication	Listed	Listed	Listed
Relational (SQL) database	Listed	Listed	Listed
Document (NoSQL) database	Not listed	Listed	Listed
File storage	Listed	Listed	Listed
Realtime	Listed	Listed	Listed
Serverless functions	Listed	Listed	Listed
Website hosting	Not listed	Listed	Listed
Messaging (push/email/SMS)	Not listed	Listed	Listed
Analytics / crash reporting	Not listed	Listed	Not listed

Read with care

"Not listed" means the feature does not appear on the pricing page. It does not mean the product lacks it. Firebase lists 9 of 9 features, Appwrite 8 and Supabase 5.

Free plans

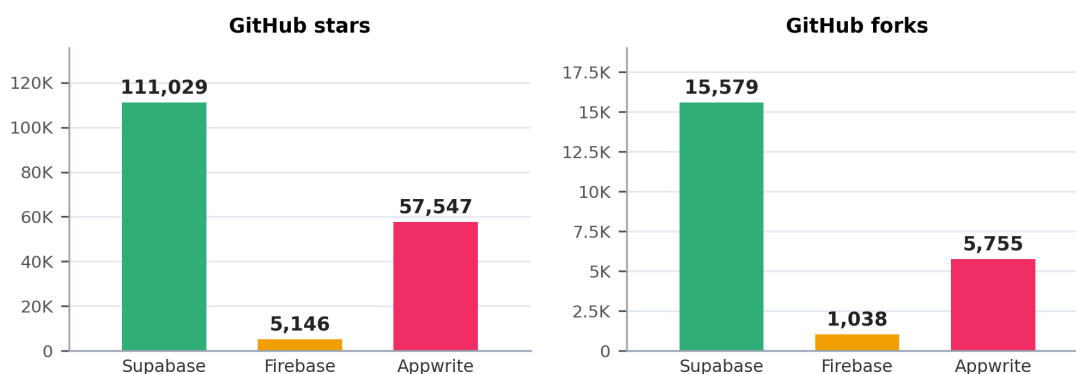


Key takeaway

Appwrite has the most generous free plan. Firebase's free plan has no Cloud Storage (file storage), but it includes other storage such as Firestore, Realtime Database and Hosting, which are not shown here.

Platform	Projects	Inactivity	Other limits
Supabase	2 active projects	Paused after 1 week of inactivity	500 MB database
Firebase	No project limit shown	No pausing shown	Cloud Storage and Cloud Functions not available; Hosting has 10 GB
Appwrite	2 projects (shared resources)	Paused after 1 week of inactivity	1 database, 1 bucket and 2 functions per project

4 User engagement (GitHub)



Key takeaway

Supabase has the most stars and forks, then Appwrite. Stars show developer interest, not paying customers.

Note

Firebase's numbers come from its JavaScript library only (firebase-js-sdk), so they are not a fair comparison with the other two.

5 Pricing recommendations

These suggestions are written for a new platform entering this market (an assumption).

1. **Offer a free plan.** All three competitors do.
2. **Price the first paid plan near \$25 per month.** Supabase and Appwrite both start there.
3. **Use a fixed fee with generous included usage.** Bills are predictable, and Appwrite's large allowance made it cheapest for the medium app.
4. **Include a lot of data transfer.** It was the biggest cost in the larger scenarios.
5. **Keep the price of extra usage low.** Supabase's low price for extra data transfer made it cheapest for the large app and helps keep heavy users.

6 Limitations and assumptions

- Only three competitors were compared.
- Costs include storage and data transfer only. Database, compute and extra user costs are not included.
- The app sizes and the "data transfer = 2 x storage" ratio are my own examples, not real customer data.
- Firebase's free data transfer was approximated as 30 GB per month (1 GB per day).
- Firebase's storage prices are for older bucket types.
- Appwrite's \$25 and Supabase's \$599 (Team) are "from" prices.
- A feature marked "not listed" is not shown on the pricing page. The product may still have it.
- Prices change often and may be different after 3 October 2026.

Appendix: code and outputs

About the code

The charts in this PDF were redrawn from the same data with larger text so they are easy to read. The original plotting code is in the Jupyter notebook.

Engagement data (GitHub API)

```
import requests
import pandas as pd

repos = ["supabase/supabase", "appwrite/appwrite", "firebase/firebase-js-sdk"]

rows = []
for repo in repos:
    r = requests.get(f"https://api.github.com/repos/{repo}", timeout=15)
    if r.status_code != 200:
        print(repo, "error", r.status_code)
        continue
    d = r.json()
    rows.append({
        "product": repo.split("/")[0],
        "stars": d["stargazers_count"],
        "forks": d["forks_count"],
        "open_issues": d["open_issues_count"],
    })
```

```

df_github = pd.DataFrame(rows)
df_github

from datetime import date

df_github["source"] = "GitHub API"
df_github["date_collected"] = date.today().isoformat()

df_github.to_csv("github_engagement.csv", index=False)
df_github

```

Pricing pages: robots.txt check

```

import requests

headers = {"User-Agent": "Mozilla/5.0"}
urls = {
    "supabase": "https://supabase.com/pricing",
    "appwrite": "https://appwrite.io/pricing",
    "firebase": "https://firebase.google.com/pricing",
}

import urllib.robotparser
from urllib.parse import urlparse

for name, url in urls.items():
    p = urlparse(url)
    r = requests.get(f"{p.scheme}://{p.netloc}/robots.txt", headers=headers, timeout=15)
    rp = urllib.robotparser.RobotFileParser()
    rp.parse(r.text.splitlines())
    print(name, "| robots.txt status:", r.status_code, "| allowed:", rp.can_fetch("", url))

```

```

supabase | robots.txt status: 200 | allowed: True
appwrite | robots.txt status: 200 | allowed: True
firebase | robots.txt status: 200 | allowed: True

```

robots.txt was downloaded with a browser-like User-Agent. Appwrite's server blocked Python's default request with a 403 error, which made the default checker wrongly report "not allowed". The real rules allow the pricing page.

Downloading the pricing pages and searching for dollar prices

```

import re
from bs4 import BeautifulSoup

pages_text = {}

for name, url in urls.items():
    resp = requests.get(url, headers=headers, timeout=15)
    soup = BeautifulSoup(resp.text, "html.parser")
    for tag in soup(["script", "style"]):
        tag.decompose()
    text = soup.get_text(" ", strip=True)
    pages_text[name] = text

    matches = list(re.finditer(r"\$\d{1,3}[\d,]*", text))
    print(f"{name}: {len(matches)} dollar prices found")

```

```

supabase: 92 dollar prices found
appwrite: 11 dollar prices found
firebase: 48 dollar prices found

```

Plan fees and usage prices

```
import pandas as pd
from datetime import date

today = date.today().isoformat()

# ---- Plan fees ----
plans = pd.DataFrame([
    ["supabase", "Free", 0, "Fixed plan fee", "supabase.com/pricing"],
    ["supabase", "Pro", 25, "Fixed plan fee + usage", "supabase.com/pricing"],
    ["supabase", "Team", 599, "Fixed plan fee + usage ('from' price)", "supabase.com/pricing"],
    ["firebase", "No-cost (Spark)", 0, "Free with fixed limits", "firebase.google.com/pricing"],
    ["firebase", "Pay as you go (Blaze)", 0, "No fixed fee, pay for usage", "firebase.google.com/pricing"],
    ["appwrite", "Free", 0, "Free with fixed limits", "appwrite.io/pricing"],
    ["appwrite", "Pro", 25, "Fixed plan fee ('from' price) + pay as you go", "appwrite.io/pricing"],
], columns=["product", "plan", "monthly_fee_usd", "pricing_model", "source"])
plans["date_collected"] = today

# ---- Usage prices ----
usage = pd.DataFrame([
    ["supabase", "File storage", "100 GB (Pro)", 0.0213, "per GB"],
    ["firebase", "File storage (Cloud Storage, legacy buckets)", "5 GB (Blaze plan)", 0.026, "per GB"],
    ["appwrite", "File storage", "150 GB (Pro)", 2.8 / 100, "per GB"],
    ["supabase", "Data transfer out", "250 GB (Pro)", 0.09, "per GB"],
    ["firebase", "Data transfer out (Cloud Storage downloads)", "1 GB/day (Blaze plan)", 0.12, "per GB"],
    ["appwrite", "Data transfer out (API bandwidth)", "2 TB (Pro)", 15 / 100, "per GB"],
    ["supabase", "Active users (MAU)", "100,000 (Pro)", 0.00325, "per user"],
    ["firebase", "Active users (Authentication)", "50,000 (some features)", None, "price not shown on page"],
    ["appwrite", "Active users (MAU)", "200,000 (Pro)", None, "price not shown on page"],
], columns=["product", "item", "free_amount", "price_after_free_usd", "unit"])
usage["date_collected"] = today

plans.to_csv("plans.csv", index=False)
usage.to_csv("usage_prices.csv", index=False)
```

Cost scenarios

```
def monthly_cost(product, storage_gb, transfer_gb):
    if product == "supabase": # Pro plan
        return 25 + max(storage_gb - 100, 0) * 0.0213 + max(transfer_gb - 250, 0) * 0.09
    if product == "appwrite": # Pro plan
        return 25 + max(storage_gb - 150, 0) * 0.028 + max(transfer_gb - 2000, 0) * 0.15
    if product == "firebase": # Blaze plan, no fixed fee
        return 0 + max(storage_gb - 5, 0) * 0.026 + max(transfer_gb - 30, 0) * 0.12

scenarios = {
    # (GB stored, GB downloaded per month)
    "Small app": (20, 100),
    "Medium app": (500, 1000),
    "Large app": (2000, 5000),
}

rows = []
for name, (stor, trans) in scenarios.items():
    for p in ["supabase", "firebase", "appwrite"]:
        rows.append([name, p, round(monthly_cost(p, stor, trans), 2)])
df_cost = pd.DataFrame(rows, columns=["scenario", "product", "monthly_cost_usd"])

df_cost = df_cost.loc[list(scenarios)]

df_cost.to_csv("cost_scenarios.csv", index=False)
```

Cost curve

```
import numpy as np

# Assumption: the app downloads 2 GB of data for every 1 GB it stores
storage_range = np.arange(0, 3001, 10)
products = ["supabase", "firebase", "appwrite"]

curve = pd.DataFrame(
    {p: [monthly_cost(p, s, 2 * s) for s in storage_range] for p in products},
    index=storage_range,
)
```

```
# Where does the cheapest platform change?
cheapest = curve.idxmin(axis=1)
changes = cheapest[cheapest != cheapest.shift()]
print("Cheapest platform changes at these storage sizes (GB):")
print(changes)
```

```
Cheapest platform changes at these storage sizes (GB):
0      firebase
110    appwrite
2210   supabase
dtype: object
```

Saving the downloaded page text

```
for name, text in pages_text.items():
    with open(f"{name}_pricing_page_text.txt", "w", encoding="utf-8") as f:
        f.write(text)
    print(name, "saved,", len(text), "characters")
```

Included resources

```
included = pd.DataFrame({
    "product": ["supabase", "firebase", "appwrite"],
    "File storage included (GB)": [100, 5, 150],
    "Data transfer included (GB/month)": [250, 30, 2000],
    "Active users included (thousands)": [100, 50, 200],
})
```

Features

```
features = pd.DataFrame({
    "Authentication": [1, 1, 1],
    "Relational (SQL) database": [1, 1, 1],
    "Document (NoSQL) database": [0, 1, 1],
    "File storage": [1, 1, 1],
    "Realtime": [1, 1, 1],
    "Serverless functions": [1, 1, 1],
    "Website hosting": [0, 1, 1],
    "Messaging (push/email/SMS)": [0, 1, 1],
    "Analytics / crash reporting": [0, 1, 0],
}, index=["supabase", "firebase", "appwrite"]).T

print(features.sum().sort_values(ascending=False))
```

```
firebase      9
appwrite       8
supabase       5
dtype: int64
```

Free plans

```
pd.set_option("display.max_colwidth", None)
free = pd.DataFrame({
    "File storage (GB)": [1, 0, 2],
    "Data transfer out (GB/month)": [5, 0, 5],
    "Active users (thousands)": [50, 50, 75],
    "Realtime connections": [200, 100, 250],
}, index=["supabase", "firebase", "appwrite"])

free_notes = pd.DataFrame([
    ["supabase", "2 active projects", "Paused after 1 week of inactivity", "500 MB database"],
    ["firebase", "No project limit shown", "No pausing shown", "Cloud Storage and Cloud Functions not available; Hosting has 10 GB"],
    ["appwrite", "2 projects (shared resources)", "Paused after 1 week of inactivity", "1 database, 1 bucket and 2 functions per project"],
], columns=["product", "projects", "inactivity", "other limits"])
```